Getting Type Knowledge to TypeScript

There will often come a time where you will know more about the type information of data in your code than TypeScript will be able to infer. TypeScript provides a few different ways for you to refine the inferred types so as to allow more accurate typing.

TypeScript allows for typecasting with the keyword `any`. The `any` type is the default where TypeScript can't infer a type, and it means that any value is legal there. You can explicitly use `any` in cases where you think that TypeScript's expected inference is likely to be too constricting.

This is perhaps most helpful when dealing with data from libraries or modules that don't use TypeScript:

```
let externalValue: any = SomeModule.someFunction()
```

The use of `any` prevents TypeScript from doing any type checking on the `externalValue` variable, which means that the return value of `someFunction` won't be used to constrain other code.

In practice, using `any` allows you to gradually add type checking to existing code by explicitly stating what data is or is not type checked. As you add more type information, often the uses of `any` can be replaced with more specific types.

Conversely, there are times when you may know the type is more specific than TypeScript assumes rather than more general.

When we know more about the type of the data than the compiler can know, TypeScript provides two ways to send that information to the compiler. The one we'll be using in this book uses the keyword `as`:

```
let elements: HTMLElement[] = document.querySelectorAll(this.targetSelector)
let inputElements: HTMLInputElement[] = elements as HTMLInputElement[]
```

An alternate syntax, which we won't use because it is confusing in React code,